

Engineering for Conformance

A Case Study in Bringing an Emulator into Line with a Real-World API

The Philips Hue API and the diyHue Bridge Emulator

Software Engineering Teaching Notes

June 28, 2026

Abstract

This chapter is a worked example of a recurring task in professional software engineering: making one program faithfully imitate another whose behaviour is defined by a *specification* rather than by shared source code. Our subject is **diyHue**, an open-source emulator of the Philips Hue smart-lighting bridge. We start from a real, automated measurement showing the emulator is only $\approx 33\%$ conformant with the Hue API, then repair nine subsystems and lift conformance to 100% of an automated check suite (and a full test suite from 44 passing / 38 failing to 110 passing / 0 failing). Each repair is presented as a small, self-contained lesson: the rule being violated, the bug, the fix, and the general principle it illustrates. The recurring themes are *input validation*, *representation invariants*, *total functions*, *idempotent reconciliation*, and *executable specifications*.

Key idea

A specification is a *contract*. When you cannot see the other party's code, the contract is all you have. "It works on my device" is not the same as "it obeys the contract", and only the second one makes independent clients interoperate.

Learning objectives

After studying this chapter you should be able to:

- explain the difference between a *test* and a *conformance scorecard*, and why both are useful;
- identify the classic defect classes—missing input validation, broken representation invariants, partial (non-total) functions, unsafe exception handling, and identity-vs-name confusion;
- apply modular arithmetic and clamping to validate bounded inputs;
- reason about ownership graphs and idempotent reconciliation;
- recognise why catching `Exception` can hide control-flow bugs in Python.

Contents

Learning objectives	1
1 Background: two APIs and one emulator	2
2 Method: measure before you mend	2
3 Case studies	3
3.1 Input validation: clamping and wrapping	3
3.2 Representation invariants: the response envelope	4
3.3 Data modelling: the device/service ownership graph	4
3.4 Event semantics: batching, cascades, and an import trap	5

3.5	Total functions: the rules-engine fall-through	6
3.6	Robustness: a crash hiding behind a missing key	6
3.7	Resource management: rate limiting	7
3.8	Idempotent reconciliation: de-duplicating MQTT sensors	7
3.9	Schema-driven dispatch: the tap-dial automation	8
4	Verification: the scorecard as a safety net	8
5	Recurring principles	8
	Exercises	9
	References	9

1 Background: two APIs and one emulator

A Philips Hue *bridge* is a small Linux device that speaks ZigBee to the lamps and HTTP to phones and home-automation software. Two generations of HTTP interface coexist:

- the **v1 (legacy) REST API** under `/api/<user>/...`, whose light state uses integer fields such as `bri` (1–254), `ct` (153–500 mireds), and `hue` (0–65535, wrapping); and
- the **CLIP v2 API** under `/clip/v2/resource/...`, a resource-oriented model where each physical product is a `device` resource that *owns* a set of *service* resources (a `light`, a `button`, a `zigbee_connectivity`, ...). Brightness here is a floating-point *percentage* (0–100).

diyHue re-implements both, plus a Server-Sent-Events (SSE) stream and a DTLS “entertainment” channel, on top of arbitrary hardware (ZigBee, MQTT, WLED, ESPHome, ...). Because real Hue apps were written and tested against a *real* bridge, the emulator only works if it reproduces the real bridge’s observable contract precisely.

Definition: conformance

Conformance is the degree to which an implementation’s observable behaviour matches a specification. It is observable: it says nothing about *how* the program is built, only about what a client can see.

2 Method: measure before you mend

Before changing code we built an automated **conformance scorecard**: a program that pokes the emulator and checks each spec rule, then prints a single weighted percentage. Crucially, a failing check does *not* abort the run; it is recorded and reported, so the scorecard always produces a number you can watch rise as you work.

Key idea

Make the invisible measurable. “Improve compliance” is not actionable. “Raise the weighted conformance score from 33% to 95%” is. A good metric turns a vague goal into a gradient you can descend.

Each check carries a severity weight (HIGH = 3, MED = 2, LOW = 1), and the overall score is

$$\text{score} = 100 \times \frac{\sum_{c \in \text{passing}} w_c}{\sum_{c \in \text{all}} w_c}.$$

Weighting by severity means fixing one crash counts for more than polishing three cosmetic fields—the metric encodes our priorities.

Table 1: Conformance before and after the work in this chapter.

	Before	After
Weighted conformance score	32.8% (grade E)	100% (grade A)
Scorecard checks passing	9/27	27/27
Full unit/contract suite	44 pass / 38 fail	110 pass / 0 fail

The scorecard is itself written with `pytest`. A non-conformant probe is marked `xfail` (“expected failure”), so the suite stays green while still recording the gap; as a bug is fixed the probe silently flips to a pass and the score rises with no edit to the check. A final *regression gate* asserts the score never falls below a floor, so a future change that breaks conformance fails the build.

Definition: executable specification

An *executable specification* is a checkable encoding of the rules a program must obey. Unlike prose, it cannot drift out of date silently: if the rule and the code disagree, the test run tells you.

3 Case studies

We now walk through the nine repairs. Each is a miniature lesson; read the “Key idea” boxes even if you skim the code.

3.1 Input validation: clamping and wrapping

What the specification says

A v1 light command must respect field ranges: `bri` ∈ [1, 254], `ct` ∈ [153, 500], `sat` ∈ [0, 254], and `hue` is a 16-bit value that *wraps* modulo 65536. `bri=0` is *not* valid (“off” is `on=false`, not brightness zero) [1, 2].

The emulator wrote whatever the client sent straight into its state, so `{"bri":999}` was stored verbatim. We added a single validation function applied on every direct state write:

```

1 V1_STATE_RANGES = {"bri": (1, 254), "ct": (153, 500), "sat": (0, 254)}
2
3 def clampV1State(state):
4     """Clamp/wrap v1 light-state values into spec-valid ranges, in
5     place."""
6     for key, (lo, hi) in V1_STATE_RANGES.items():
7         if key in state and isinstance(state[key], (int, float)) \
8             and not isinstance(state[key], bool):
9             state[key] = max(lo, min(hi, int(state[key])))
10    if "hue" in state and isinstance(state["hue"], (int, float)):
11        state["hue"] = int(state["hue"]) % 65536      # wrap, do not
12        clamp
13    ...
14    return state

```

Two details deserve attention. First, `hue` is *wrapped*, not clamped: the original code did `state["hue"] -= 65535`, which is both off-by-one (the period is 65536, not 65535) and wrong

for large increments. The correct operation is a single modulo:

$$\text{hue}' = (\text{hue} + \Delta) \bmod 65536.$$

Second, `bool` is a subclass of `int` in Python, so `isinstance(True, int)` is `True` [6]; we exclude booleans explicitly so that `on=True` is never mistaken for a numeric field.

Common pitfall

Trusting the client. Never assume input is in range just because a well-behaved client would send valid values. A different client—or a malicious one—will not. Validate at the boundary, once, in a single place you can point to.

3.2 Representation invariants: the response envelope

What the specification says

Every CLIP v2 response is an object with *both* an `errors` array and a `data` array—even on error (then `data` is empty) and even on success (then `errors` is empty) [2, ?].

The emulator’s PUT handler returned only `data`; its error path returned only `errors` and even deleted the `data` key. A client that unconditionally reads `response["data"]` would crash. The fix is to make the shape an *invariant*—always present, regardless of outcome:

```
1 # success
2 response = {"errors": [], "data": [{"rid": resourceid, "rtype":
    resource}]}
3 # error
4 return {"errors": [{"description": f"...{resource}"}], "data": []}, 500
```

Definition: representation invariant

A *representation invariant* is a property that every value of a type must satisfy. “A v2 response always has `errors` and `data` arrays” is one. Enforcing invariants at the point of construction means clients never have to defend against malformed values.

A sibling bug lived in the “get one button by id” path, which returned `data: [object.getButtons()]`—a *list inside a list*, because `getButtons()` already returns a list. The repair flattens the result and, because a single button is a sub-resource of a sensor device, resolves it by searching for the one matching id:

```
1 if resource in ["button", "relative_rotary"]:
2     method = "getButtons" if resource == "button" else "getRotary"
3     items = [item for sensor in bridgeConfig["sensors"].values()
4              for item in getattr(sensor, method)()
5              if item["id"] == resourceid]
6     return {"errors": [], "data": items}
```

3.3 Data modelling: the device/service ownership graph

The v2 model is a small graph: a `room` has *children* (the devices in it) and *services* (its `grouped_light`); a `device` owns its services; a `grouped_light` is owned by its room or zone [2, ?]. Three bugs lived here.

(a) **Missing owner.** Every service and device must carry an `owner` reference `{rid, rtype}`. Devices omitted it, failing schema validation. We added it at each device’s single construction point.

(b) **Non-light children were dropped.** A room could contain switches and sensors, but `getV2Room` only emitted its lights. We gave `Group` a `device_children` list that is initialised from stored data (as a *copy*, so later mutation of the source cannot leak in), rendered into the children array, de-duplicated against the lights, and persisted on save:

```
1 self.device_children = list(data["device_children"]) \
2     if "device_children" in data else []
3 ...
4 seen = set()
5 for light in self.lights:
6     if light():
7         rid = light().getDevice()["id"]
8         if rid not in seen:
9             seen.add(rid); result["children"].append({"rid": rid, "
              rtype": "device"})
10 for rid in self.device_children:                # non-light devices,
    deduped
11     if rid not in seen:
12         seen.add(rid); result["children"].append({"rid": rid, "rtype":
            "device"})
```

(c) **Wrong owner on grouped_light.** It claimed to be owned by a *device*; the spec says a grouped light is owned by its room/zone (or, for group 0, by `bridge_home`). We compute the owner from the group’s type.

Key idea

Model identities, not names. A reference points at an `id`, never at a display name. Names change; identities do not. Half the bugs in this section are a node pointing at the wrong identity.

3.4 Event semantics: batching, cascades, and an import trap

What the specification says

A single light change is delivered as *one* SSE event whose `data` array contains every affected resource: the light, its device, and the `grouped_light` rollups for every owning room/zone and for `bridge_home` (the “all lights” group) [4, ?].

The emulator emitted a *separate* event per resource and never sent the group rollups, so a dashboard tracking a room’s aggregate brightness silently went stale. We rewrote the emitter to build one batched event and append the rollups for every group containing the light.

This subsection carries a subtle and very instructive Python lesson. To find the owning groups, the light needs the global configuration. A naive `import configManager` *inside* the function re-runs that module’s top-level code, which parses command-line arguments and calls `sys.exit()` when run under the test harness. That raises `SystemExit`—which is **not** a subclass of `Exception` [5]:

```
BaseException ⊃ {SystemExit, KeyboardInterrupt, Exception ⊃ KeyError, ...}
```

so a surrounding `except Exception` does *not* catch it and the whole test crashes. The fix never triggers a fresh import; it reads the *already-imported* module out of the module cache:

```
1 import sys
2 groups = {}
3 cfgMod = sys.modules.get("configManager")    # do NOT 'import' here
4 if cfgMod is not None:
5     try:
6         groups = cfgMod.bridgeConfig.yaml_config.get("groups", {})
7     except Exception:
8         groups = {}
```

Common pitfall

import runs code. In Python, importing a module executes its top level. Importing for a side effect deep inside a hot function can re-run that code in a surprising context. And remember: `except Exception` does *not* catch `SystemExit` or `KeyboardInterrupt`—those are `BaseExceptions` by design [5], so a stray `sys.exit()` sails right past your handler.

3.5 Total functions: the rules-engine fall-through

The v1 rules engine evaluates conditions of the form `{address, operator, value}`. The dispatch was a chain of `if/elif` on the operator, with *no final else*. An operator the engine did not recognise—"not in", "stable"—matched no branch, so the loop simply fell through and the condition was treated as *satisfied*. A rule that should never fire could therefore fire.

Key idea

Make dispatch total. A function that switches on a value should handle *every* value, including the unexpected ones. The safe default for an unknown operator is “condition not met”, never “silently true”. In practice this means: always write the final `else`.

```
1 elif condition["operator"] == "not in":
2     ...                                # genuinely evaluate the time
3     window
4 elif condition["operator"] in ("stable", "not stable"):
5     return [False, 0]                  # cannot evaluate → do not fire
6 else:
7     logging.debug("unsupported operator " + str(condition["operator"]))
8     return [False, 0]                  # unknown → safe default
```

3.6 Robustness: a crash hiding behind a missing key

Creating a v2 `behavior_instance` with a perfectly valid payload returned HTTP 500. The cause was one line that assumed an optional field always exists:

```
1 for resource in self.configuration["where"]:    # KeyError if
2     absent
```

The repair is a one-character class of fix—ask for the key *defensively*:

```
1 for resource in self.configuration.get("where", []):
```

Common pitfall

`d[k]` demands that `k` exists; `d.get(k, default)` tolerates its absence. Use indexing only when absence is genuinely a bug worth crashing for. For optional fields, supply a default.

3.7 Resource management: rate limiting

What the specification says

A real bridge processes only about *ten* light commands per second; beyond that it returns HTTP 429 (“Too Many Requests”) [3, 1]. Apps are written to expect this back-pressure.

The emulator applied every command immediately, so a client tuned to the real bridge could overwhelm slow hardware. We added a *sliding-window* limiter: keep the timestamps of recent commands, drop those older than one second, and reject once the window is full.

```
1 _RATE_LIMITS = {"light": 10}
2 _rate_state = {}
3
4 def rateLimited(resource):
5     limit = _RATE_LIMITS.get(resource)
6     if not limit:
7         return False
8     now = monotonic()
9     window = _rate_state.setdefault(resource, [])
10    window[:] = [t for t in window if t > now - 1.0]    # forget old
11                  events
12    if len(window) ≥ limit:
13        return True
14    window.append(now)
15    return False
```

Definition: sliding-window rate limit

A *sliding-window* limiter permits at most N events in any trailing time window of length T . It is implemented by remembering recent event times and discarding those older than T before each decision. Note the use of a *monotonic* clock, which never jumps backwards when the system time is [7] adjusted.

3.8 Idempotent reconciliation: de-duplicating MQTT sensors

When a ZigBee device is renamed in the Zigbee2MQTT layer, the bridge re-announces it under a new friendly name. The emulator keyed sensors by *name*, so a rename created a *second* copy instead of updating the first—and the two copies fought over the device. The real identity of a ZigBee device is not its name but its `ieeeAddr` (its hardware address). The fix groups sensors by that stable key, keeps exactly one per logical type, and renames in place:

```
1 def getSensorsByIeeeAddr(ieeeAddr):
2     return [s for s in bridgeConfig["sensors"].values()
3             if getattr(s, "protocol", None) == "mqtt"
4             and s.protocol_cfg.get("ieeeAddr") == ieeeAddr]
```

Key idea

Reconcile, don't duplicate. Processing the same announcement twice should leave the system in the same state as processing it once. This property is called *idempotence*, and the way to get it is to key your records by a stable identity and *update* rather than *insert*.

3.9 Schema-driven dispatch: the tap-dial automation

The most involved repair re-implemented the automation engine for the RDM002 “tap dial” (four buttons plus a rotary). The button-event integer encodes both *which* button and *what* happened:

$$\underbrace{1}_{\text{button 1}} \underbrace{002}_{\text{short release}}$$

i.e. `button = event // 1000` and `action = event % 1000`. We map the trailing digit to a named action through a small table—clearer and more total than a chain of ifs:

```
1 BUTTON_ACTIONS = {0: "on_initial_press", 1: "on_repeat",
2                   2: "on_short_release", 3: "on_long_press"}
3 ...
4 buttonKey = "button" + str(buttonevent // 1000)
5 actionKey = BUTTON_ACTIONS.get(buttonevent % 1000)
```

The rotary dial is a separate logical sensor whose automation is configured on its *parent* switch; matching therefore checks both the device's own id and its `parent_id_v2`. The new schema lives alongside the legacy one (we fall back to the old format when the new keys are absent), so existing stored automations keep working—a small but important act of *backward compatibility*.

4 Verification: the scorecard as a safety net

Every repair was confirmed by re-running the scorecard and watching the number climb: 32.8% → 62.3% → 96.7% → 100%. Because the checks are executable, they double as regression protection: the gate `test_scorecard_meets_baseline` now fails the build if conformance drops below 95%.

Key idea

A fix without a test is a rumour. Each defect in this chapter is now pinned by a check that would have caught it. The scorecard is not scaffolding to be discarded—it is the executable definition of “done”.

5 Recurring principles

1. **Measure first.** Turn “be more compliant” into a number, then make the number go up.
2. **Validate at the boundary.** Clamp and wrap untrusted input once, in one place.
3. **Enforce invariants at construction.** If every response has the same shape, no client has to defend against malformed ones.
4. **Make functions total.** Handle every case; the safe default for the unknown case is rarely “succeed”.
5. **Key by identity, not by name.** Stable identifiers make reconciliation idempotent.
6. **Know your language.** `import` runs code; `SystemExit` is not an `Exception`; `bool` is an `int`.

Exercises

1. **(Wrapping.)** Show that `(hue +  $\Delta$ ) % 65536` gives the correct result for $\Delta = -90000$, whereas a single `+= 65536` correction does not. For which range of Δ does the single-correction approach work?
2. **(Invariants.)** A teammate proposes returning bare `[]` for an empty collection instead of `{"errors": [], "data": []}`. Give a concrete client that breaks, and state the invariant being violated.
3. **(Totality.)** The rules engine fell through on unknown operators. Write a property-based test that feeds random operator strings and asserts the condition is never reported as satisfied unless the operator is one of the supported set.
4. **(Rate limiting.)** The limiter uses `monotonic()` rather than `time()`. Construct a scenario (an NTP step, a daylight-saving change) where using wall-clock time would let a burst through or block a legitimate command.
5. **(Idempotence.)** Prove that processing the same `bridge/devices` announcement twice leaves the sensor table identical to processing it once, given the de-duplication rule “one sensor per type, keyed by `ieeeAddr`”.
6. **(Design.)** The tap-dial decodes `event // 1000` and `event % 1000`. What goes wrong if a future device emits button index 10? Propose an encoding that does not have this ceiling and discuss its compatibility cost.

References

- [1] Signify / Philips Hue. *Hue Developer Documentation — CLIP API v2 and legacy API, core concepts (resource model, rate limits, light state ranges)*. <https://developers.meethue.com/develop/hue-api-v2/>. (Access requires a free developer account; the machine-readable transcription in [2] mirrors its field names and enums.)
- [2] OpenHue. *openhue-api — an OpenAPI 3 specification of the Philips Hue CLIP v2 API*. <https://github.com/openhue/openhue-api>. Source of the device/service composition model, the `{errors,data}` envelope, and the `dimming.brightness` 0–100 percentage.
- [3] R. Fielding and M. Nottingham. *RFC 6585: Additional HTTP Status Codes*. IETF, April 2012. <https://datatracker.ietf.org/doc/html/rfc6585>. Defines 429 Too Many Requests and the `Retry-After` header.
- [4] WHATWG. *HTML Living Standard, §9.2 Server-sent events*. <https://html.spec.whatwg.org/multipage/server-sent-events.html>. The `text/event-stream` wire format (`id:/data:` fields) and the `EventSource` API.
- [5] Python Software Foundation. *Python Language Reference — Built-in Exceptions: the exception hierarchy*. <https://docs.python.org/3/library/exceptions.html>. `SystemExit` and `KeyboardInterrupt` derive from `BaseException`, not `Exception`, so `except Exception` does not catch them.
- [6] Python Software Foundation. *The Python Standard Library — Numeric Types and Boolean Values*. <https://docs.python.org/3/library/stdtypes.html#boolean-type-bool>. `bool` is a subclass of `int` (`True=1`, `False=0`; see also PEP 285).

- [7] Python Software Foundation. *The Python Standard Library — `time.monotonic()`*. <https://docs.python.org/3/library/time.html#time.monotonic>. A clock that cannot go backwards and is unaffected by system-clock adjustments (rationale in PEP 418).
- [8] The diyHue project. *diyHue — a Philips Hue bridge emulator (source repository)*. <https://github.com/diyhue/diyHue>. Primary source for the emulator behaviour discussed here; specific defects cite `BridgeEmulator/...` by file and line.
- [9] Companion artifacts in this repository: `docs/spec-compliance-audit.md` (the full defect inventory), `docs/real-bridge-research.md` (measured behaviour of the genuine BSB002 bridge), and `tests/test_spec_compliance.py` (the executable scorecard).

These notes accompany the change set on the `spec-compliance-fixes` branch of diyHue. The companion documents `spec-compliance-audit.md` (the full defect list) and `real-bridge-research.md` (how the real bridge behaves) provide the underlying evidence.